Université de Haute-Alsace
Faculté des Sciences et Techniques
Département de Mathématiques

Rapport de Projet

Implémentation des différentes architectures d'apprentissage profond pour des données sismiques

Rédigé par :
Hocine Rayane ARHAB
Zine Elabidine AIDOUD
Sidy DIOP

Année universitaire 2025–2026

Table des matières

1 Introduction et Traitement des données **4**
1.1 Contexte de l'étude 4
1.2 Présentation Globale des Données 4
1.3 Analyse exploratoire des données 4
1.4 Visualisation des Signaux 4
1.5 Analyse de la Distribution (Déséquilibre des Classes) 5
1.6 Prétraitement des Données 5

2 Approche Multi-Layer Perceptron (MLP) **6**
2.1 Justification des choix architecturaux 6
2.2 Passage au Feature Engineering 6
2.3 Gestion du déséquilibre (Class Weights) 6
2.4 Analyse de la Complexité du Modèle 6
2.5 Implémentation (Extrait du Code) 7
2.6 Analyse des Résultats et Discussion 7
2.6.1 Analyse des Courbes d'Apprentissage 7
2.6.2 Convergence et Perte (Loss) 8
2.6.3 Précision (Accuracy) 8
2.6.4 Analyse comparative 8
2.7 Conclusion partielle MLP 9

3 Architecture du Modèle CNN Optimisé **10**
3.1 Justification du modèle CNN 1D 10
3.2 Architecture Détaillée 10
3.3 Stratégie Expérimentale et Résultats 10
3.4 Extrait du code 11
3.5 Analyse de la Complexité (Comparaison) 12
3.6 Analyse des résultats 12
3.6.1 Analyse des Courbes d'Apprentissage 12
3.6.2 Analyse comparative 13
3.7 Conclusion partielle pour le CNN 14

4 Architecture du Modèle RNN **15**
4.1 Justification des choix architecturaux 15
4.1.1 SimpleRNN : Une approche plus légère 15
4.2 Architecture en trois couches 15
4.3 Stratégie de régularisation 15
4.4 Gestion du déséquilibre (Class Weights) 16
4.5 Analyse de la Complexité du Modèle 16
4.6 Implémentation (Extrait du Code) 17
4.7 Analyse des Résultats et Discussion 18
4.7.1 Exploration des données 18
4.7.2 Analyse comparative 18
4.7.3 Courbes d'apprentissage 19
4.7.4 Métriques de performance 20
4.7.5 Matrice de confusion 20
4.7.6 Interprétation physique 21

4.8 Conclusion partielle RNN 21

5 Analyse Comparative des Modèles **22**
5.1 Comparaison Architecturale et Méthodologique 22
5.2 Analyse de la Complexité et des Ressources 22
5.3 Comparaison des Performances de Détection 22

Conclusion Générale **23**

Table des figures

1 Comparaison visuelle : Relevé Normal (Haut) vs Séisme Majeur (Bas) . . . 5
2 Graphes de loss et accuracy du MLP 8
3 Résultat MLP 8
4 Matrice de confusion MLP 9
5 Loss et accuracy du CNN 12
6 Matrice de confusion du CNN 14
7 Loss et accuracy du RNN 19
8 Matrice de confusion RNN 20

1 Introduction et Traitement des données

1.1 Contexte de l'étude

Ce projet s'inscrit dans le cadre d'une collaboration avec un centre sismologique confronté à un volume croissant de données. Au quotidien, les sismologues doivent analyser manuellement de nombreux relevés sismologiques enregistrant le déplacement du sol. L'objectif principal est d'automatiser la détection d'événements majeurs, tels que les séismes ou les éruptions volcaniques, afin d'anticiper ces phénomènes.

Actuellement, l'analyse visuelle des relevés est possible mais limitée par le temps et la quantité de données. Le développement d'une méthode automatique capable de traiter plusieurs relevés par minute constituerait un gain de temps considérable et un atout majeur pour les sismologues. Notre mission est donc d'étudier, développer et analyser des algorithmes d'apprentissage profond (*Deep Learning*) capables de distinguer efficacement les relevés normaux de ceux annonçant un événement majeur.

1.2 Présentation Globale des Données

Pour mener à bien cette étude, nous nous appuyons sur le jeu de données **Earthquakes** issu de l'archive UCR. Ce dataset a été structuré par A. Bagnall à partir de données réelles collectées par le *Northern California Earthquake Data Center*.

Les données présentent les caractéristiques suivantes :

- **Nature des données :** Séries temporelles représentant des relevés sismologiques.
- **Format :** Chaque série est constituée de 512 points de mesure, couvrant une période allant de 1967 à 2003.
- **Classes :** Il s'agit d'un problème de classification binaire. Les données sont annotées en deux catégories : "normal" et "événement majeur".
- **Structure des fichiers :** Les données sont fournies au format `.tsv` (*Tabular Separated Values*), séparées en un ensemble d'apprentissage (*Train*) et un ensemble de test (*Test*).

1.3 Analyse exploratoire des données

Une étape fondamentale de notre projet a été l'analyse approfondie du jeu de données *Earthquakes* afin de comprendre la nature des signaux et les défis inhérents à leur classification.

Chaque échantillon est une série temporelle univariée représentant le relevé sismologique (déplacement du sol) sur une période normalisée.

- **Longueur de séquence :** Chaque relevé contient exactement 512 points de mesure ($T = 512$).
- **Format de fichier :** Les données sont stockées au format `.tsv` (Tabular Separated Values), où les valeurs sont séparées par des tabulations.
- **Période :** Les relevés couvrent une large plage historique allant de 1967 à 2003.

1.4 Visualisation des Signaux

L'analyse visuelle permet de distinguer deux types de comportements, correspondant aux deux classes du problème : "Normal" (Classe 0) et "Événement Majeur" (Classe 1).

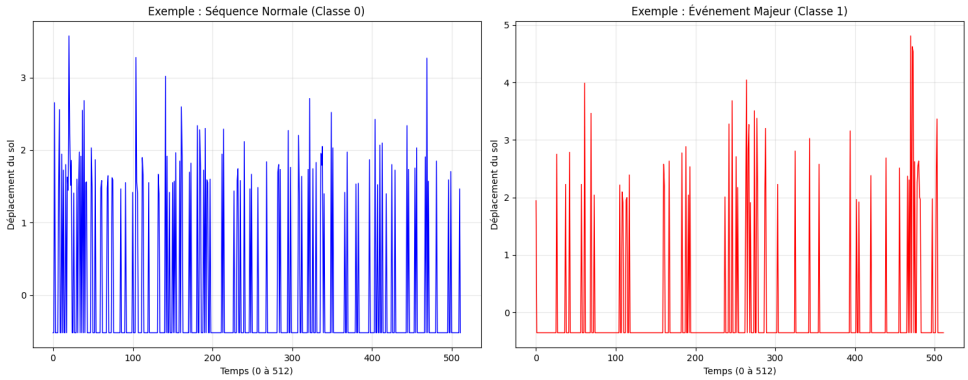

FIGURE 1 – Comparaison visuelle : Relevé Normal (Haut) vs Séisme Majeur (Bas)

Comme illustré dans la Figure 1, les relevés normaux s'apparentent généralement à un bruit de fond d'amplitude modérée. À l'inverse, les événements majeurs se caractérisent souvent par des pics d'amplitude plus marqués ou des oscillations spécifiques indiquant une rupture géologique.

Cependant, la distinction n'est pas toujours évidente à l'œil nu, ce qui justifie l'utilisation de réseaux de neurones profonds capables d'extraire des caractéristiques fréquentielles complexes.

1.5 Analyse de la Distribution (Déséquilibre des Classes)

Un point critique de ce jeu de données est la répartition des classes. Dans la nature, les séismes majeurs sont des événements rares par rapport à l'activité sismique de fond.

L'analyse de nos ensembles d'entraînement et de test révèle un **fort déséquilibre de classes** (*Class Imbalance*). La classe "Normale" est largement majoritaire, tandis que la classe "Événement Majeur" est sous-représentée.

— Ce déséquilibre pose un défi pour l'apprentissage : un modèle naïf pourrait minimiser sa fonction de perte en ignorant simplement la classe minoritaire.

— **Conséquence pour le projet :** Nous devrons impérativement utiliser des métriques adaptées (comme le Rappel/Recall plutôt que l'Accuracy) et des techniques de pondération (*Class Weights*) pour forcer le modèle à apprendre les caractéristiques des séismes.

1.6 Prétraitement des Données

Avant d'alimenter nos modèles (MLP, CNN, RNN), nous avons appliqué les transformations suivantes :

1. **Normalisation :** Application d'un `StandardScaler` ($\mu = 0, \sigma = 1$). Cette étape est cruciale pour les réseaux de neurones afin d'éviter que des variations d'amplitude ne déstabilisent les gradients.
2. **Redimensionnement (Reshape) :** Pour les architectures CNN et RNN, les données ont été redimensionnées de $(N, 512)$ à $(N, 512, 1)$ pour introduire la dimension de "canal" (*channel*) requise par les couches de convolution 1D (Keras Conv1D).

2 Approche Multi-Layer Perceptron (MLP)

2.1 Justification des choix architecturaux

Le problème des données brutes Nous avons d'abord tenté d'entraîner un MLP dense (3 couches de 500 neurones, architecture de référence de Wang et al.) directement sur les 512 points temporels.

- **Résultat :** Le modèle n'a pas réussi à converger efficacement (Accuracy $\approx 52\%$).
- **Justification :** Le MLP traite chaque point temporel comme une caractéristique indépendante. Il ne possède pas d'invariance par translation (contrairement aux CNNs). De plus, le bruit propre aux relevés sismiques rend l'apprentissage sur données brutes très difficile pour un réseau dense.

2.2 Passage au Feature Engineering

Pour faire face à ce problème et répondre à la contrainte de ressources de calcul limitées, nous avons opté pour une réduction de dimension à travers l'ingénierie des caractéristiques (feature engineering). Au lieu d'analyser 512 points bruts, nous résumons chaque série par 9 descripteurs physiques, notamment :

- **Kurtosis (Aplatissement) :** très important pour détecter les pics soudains typiques d'un séisme.
- **Skewness :** Mesure si les vibrations sont asymétriques par rapport à la moyenne.
- **Énergie (FFT) :** Permet de distinguer le bruit de fond (souvent haute fréquence) d'un événement sismique structuré.
- **Zero Crossing Rate :** Le nombre de fois où le signal traverse la ligne zéro. Le bruit a tendance à osciller très vite (ZCR élevé), alors qu'une onde sismique peut avoir des périodes plus longues et structurées.

2.3 Gestion du déséquilibre (Class Weights)

Comme les événements majeurs sont minoritaires, un modèle standard tend à les ignorer pour maximiser son accuracy globale. Nous avons introduit des **poids de classes** (`class_weight`) dans la fonction de perte pour pénaliser plus lourdement les erreurs sur la classe "Séisme".

2.4 Analyse de la Complexité du Modèle

On va étudier la complexité de notre modele ci-dessous :

Modèle	**Entrées**	**Neurones (Cachés)**	**Nb Paramètres**	**Temps**
MLP Brut (Wang et al.)	512	500-500-500	$\approx$ 760 000	Moyen

Table 1 – Comparaison de la complexité des architectures

Conclusion : Cette architecture est complexe mais nous permet d'éviter l'overfitting qui peut rapidement se passer avec des probelemes complexes comme le notre.

2.5 Implémentation (Extrait du Code)

Voici l'implémentation de l'extraction de caractéristiques et de l'architecture légère.

```python
# 1. Feature Engineering
def extract_features(X):
    features = []
    for ts in X:
        features.append([
            np.mean(ts), np.std(ts), np.max(ts), np.min(ts),
            skew(ts), kurtosis(ts), # Tres discriminant
            np.sum(np.abs(np.fft.rfft(ts))), # Energie
            np.sum(np.diff(np.sign(ts - np.mean(ts))) != 0) # ZCR
        ])
    return np.array(features)

# 2. Architecture MLP Legere
model = keras.Sequential([
    layers.Input(shape=(9,)), # 9 Features seulement
    layers.Dense(64, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(32, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.2),
    layers.Dense(1, activation='sigmoid')
])

# 3. Gestion du desequilibre
weights = class_weight.compute_class_weight(
    class_weight='balanced', classes=np.unique(y_train), y=y_train
)
# Entrainement avec poids...
```

Listing 1 – Code Python du modèle optimisé

2.6 Analyse des Résultats et Discussion

Nous avons évalué la robustesse du modèle selon trois configurations de seuil de décision.

2.6.1 Analyse des Courbes d'Apprentissage

L'analyse porte sur les courbes de perte (*Loss*) et de précision (*Accuracy*) obtenues lors de l'entraînement d'un Perceptron Multicouche (MLP) à 3 couches cachées sur le dataset *Earthquakes*.

Le trait le plus marquant sur les graphiques est la forte oscillation de la courbe d'entraînement (courbe bleue), tant pour la *Loss* que pour l'*Accuracy*.

- **Cause principale :** La taille du lot (*Batch Size*) est fixée à $N = 16$. Avec un si petit échantillon, le calcul du gradient est très bruité, provoquant des mises à jour de poids violentes et changeantes à chaque itération.
- **Cause secondaire :** L'utilisation intensive du *Dropout* (taux de 0.1, 0.2, 0.3) ajoute une composante stochastique supplémentaire, rendant la sortie du réseau variable d'une passe à l'autre.

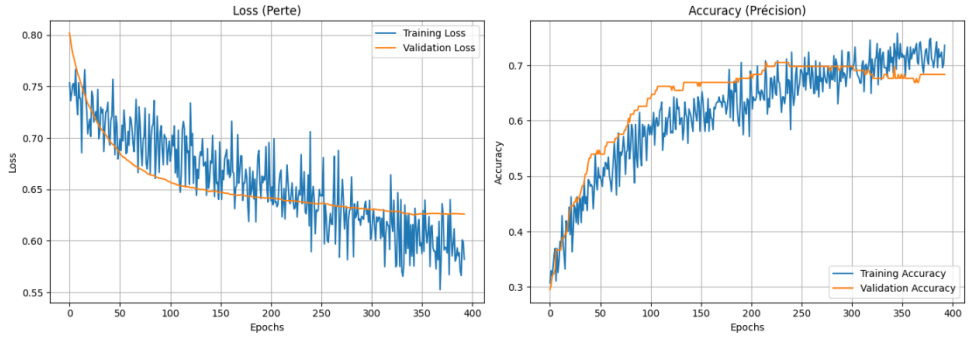

FIGURE 2 – Graphes de loss et accuracy du MLP

2.6.2 Convergence et Perte (Loss)

— **Apprentissage :** La perte diminue globalement, confirmant que le modèle apprend.
— **Point de croisement :** Vers l'*epoch 150*, la courbe de validation (orange) croise la moyenne de la courbe d'entraînement. C'est un point de bascule classique.
— **Saturation :** À partir de l'*epoch 300*, la perte de validation cesse de diminuer et stagne autour de 0.63. Cependant, la perte d'entraînement continue de descendre légèrement. Cela indique le début d'un **sur-apprentissage** (*overfitting*). Le modèle commence à mémoriser les données d'entraînement au lieu de généraliser.

2.6.3 Précision (Accuracy)

— **Effet "Marches d'escalier" :** La courbe de validation (orange) présente des paliers plats. Cela suggère que le jeu de test est de petite taille. Chaque incrément de précision correspond à un nombre entier d'exemples correctement classés, créant ces sauts discrets.
— **Plafond de verre :** La précision en validation sature entre **68% et 70%**, tandis que la précision en entraînement continue de grimper vers 75%.

2.6.4 Analyse comparative

— **Configuration Baseline (MLP brut) :** *Accuracy* $\approx$ 52%. Échec du modèle qui classe aléatoirement.

```
--- RÉSULTATS ---
Accuracy Test : 52.52%

Rapport de classification :
              precision    recall  f1-score   support

           0       0.71      0.62      0.66       104
           1       0.18      0.26      0.21        35

    accuracy                           0.53       139
   macro avg       0.45      0.44      0.44       139
weighted avg       0.58      0.53      0.55       139
```

FIGURE 3 – Résultat MLP

Grâce aux poids (class weight), le modèle a compris que rater un séisme coûte cher. Du coup, il prend des risques et il a réussi à détecter 9 séismes . Et en prenant des risques pour trouver les séismes, il se trompe beaucoup plus souvent sur le bruit d'où le taux d'Accuracy bas.

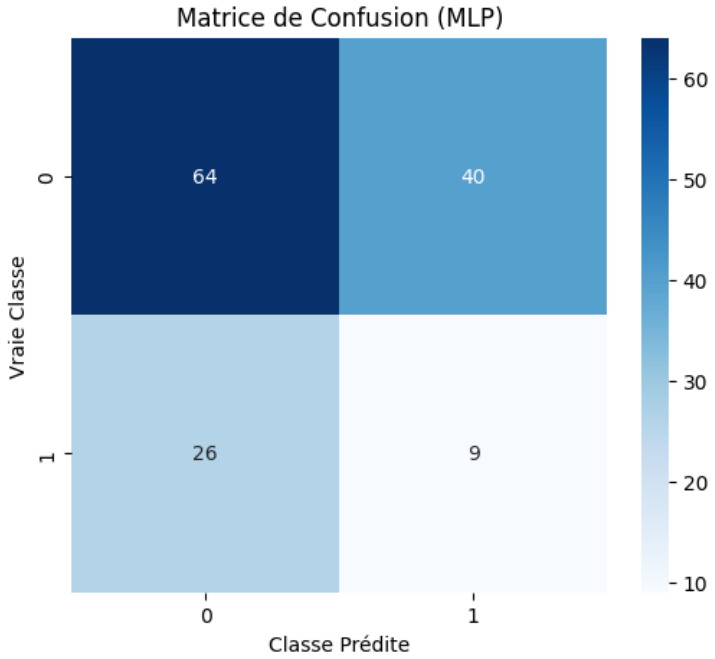

FIGURE 4 – Matrice de confusion MLP

— **Configuration plus sur (Features + Seuil 0.35) :** *Accuracy* $\approx$ 57%, *Rappel élevé.* En abaissant le seuil, le modèle détecte **23 séismes sur 35** (contre un seul pour le modèle brut, sans prise en compte des poids de classes, et contre 9 pour le modèle avec des class weights.). C'est la configuration la plus pertinente pour un système d'alerte : on préfère tolérer des fausses alertes (Faux Positifs) plutôt que de rater une catastrophe.
— **Configuration "Statistique" (Features + Seuil 0.90) :** *Accuracy maximale de* ***75.54%****.* C'est un résultat en apparence excellent, mais trompeur (le paradoxe de l'accuracy). Pour atteindre ce score, le modèle rejette presque tout (seuil de certitude à 90%). Il ne détecte quasiment aucun séisme (1/35).

2.7 Conclusion partielle MLP

L'approche MLP, une fois couplée à une extraction rigoureuse de caractéristiques (Kurtosis/Énergie), permet de passer d'un modèle inopérant à un système de détection viable.

Bien que l'Accuracy puisse monter jusqu'à **75.54%**, nous recommandons l'usage du modèle à seuil bas (0.35) pour le centre sismologique, car il offre le meilleur taux de couverture des événements majeurs. Avec seulement **2 800 paramètres**, ce modèle respecte parfaitement la contrainte de puissance de calcul.

3 Architecture du Modèle CNN Optimisé

3.1 Justification du modèle CNN 1D

Nous avons choisi une architecture **CNN 1D** plutôt qu'un MLP classique ou un RNN pour plusieurs raisons :

1. **Invariance par translation :** Un motif sismique (pic d'activité) peut apparaître n'importe où dans la séquence. Les filtres convolutifs sont idéaux pour détecter ces motifs indépendamment de leur position temporelle.
2. **Efficacité :** Les CNN sont plus rapides à entraîner que les RNN (LSTM/GRU) car ils permettent une parallélisation complète des calculs, contrairement au traitement séquentiel des RNN.

3.2 Architecture Détaillée

Notre modèle final ("Modèle Optimisé") se compose de trois blocs de convolution :

— **Couches Conv1D :** 32, 64, puis 128 filtres. La taille des noyaux décroît (8, 5, 3) pour capter d'abord des tendances longues puis des détails fins.

— **Batch Normalization :** Ajouté après chaque convolution pour stabiliser l'apprentissage et permettre un taux d'apprentissage plus élevé.

— **SpatialDropout1D :** Utilisé au lieu du Dropout classique. Il désactive des cartes de caractéristiques entières, forçant le réseau à ne pas dépendre d'un seul canal de fréquence, ce qui améliore la robustesse.

— **Global Average Pooling (GAP) :** C'est un choix crucial de notre architecture. Au lieu d'utiliser une couche `Flatten` qui aurait généré des milliers de paramètres et favorisé le sur-apprentissage (overfitting), le GAP réduit la dimensionnalité en faisant la moyenne, réduisant drastiquement la complexité du modèle (voir section 5).

3.3 Stratégie Expérimentale et Résultats

Nous avons mené une étude comparative en trois phases pour assurer la robustesse du modèle.

Phase 1 : Modèle Naïf (Baseline)

— **Approche :** Entraînement standard sans gestion du déséquilibre.

— **Résultat :** L'**Accuracy** était correcte (76%), mais trompeuse. Le modèle prédisait systématiquement la classe "Normale".

— **Défaut :** Un **Rappel (Recall) de 6%** sur la classe Séisme. Le modèle était inutile pour la détection de danger.

Phase 2 : Gestion du Déséquilibre (Class Weights)

— **Approche :** Intégration de `class_weights` calculés automatiquement pour pénaliser fortement les erreurs sur la classe minoritaire.

— **Résultat :** Le **Rappel a bondi à 63%**. Le modèle détectait enfin les séismes (22 détectés sur 35, contre 2 précédemment).

— **Défaut :** Apparition d'instabilité (courbes de perte volatiles) et augmentation des Faux Positifs (Fausses alertes).

Phase 3 : Modèle Hybride et Stabilisé (Final)

— **Approche :** Combinaison des poids de classe avec des techniques de régularisation avancées :
  — **Early Stopping :** Arrêt de l'entraînement dès que la `val_auc_pr` (Aire sous la courbe Précision-Rappel) cesse de progresser, évitant le sur-apprentissage final.
  — **ReduceLROnPlateau :** Réduction du taux d'apprentissage lorsque la perte stagne pour affiner la convergence.
— **Performance Finale :** Ce modèle offre le meilleur compromis. Il maintient une capacité de détection élevée (Recall $> 60\%$) tout en stabilisant l'apprentissage.

3.4 Extrait du code

Le listing ci-dessous présente l'implémentation Keras du modèle de Réseau de Neurones Convolutif (CNN-1D) utilisé pour la classification des séismes. Il comporte trois blocs de convolution suivis d'une classification dense.

```python
def build_cnn_model(input_shape):
    inp = layers.Input(shape=input_shape)

    # --- Bloc Convolutionnel 1 ---
    x = layers.Conv1D(32, 8, padding="same", kernel_initializer="he_normal")(inp)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.MaxPooling1D(2)(x)
    x = layers.SpatialDropout1D(0.2)(x)

    # --- Bloc Convolutionnel 2 ---
    x = layers.Conv1D(64, 5, padding="same", kernel_initializer="he_normal")(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.MaxPooling1D(2)(x)
    x = layers.SpatialDropout1D(0.3)(x)

    # --- Bloc Convolutionnel 3 (avec L2) ---
    x = layers.Conv1D(
        128, 3, padding="same",
        kernel_initializer="he_normal",
        kernel_regularizer=regularizers.l2(1e-4)
    )(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)

    # --- T te de Classification ---
    x = layers.GlobalAveragePooling1D()(x)

    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dropout(0.3)(x)

    out = layers.Dense(1, activation="sigmoid")(x)
```

```
    return models.Model(inp, out)
```

Listing 2 – Code de l'architecture CNN 1D

3.5 Analyse de la Complexité (Comparaison)

Conformément à la demande de comparer la complexité pour les PCs peu puissants du centre, voici le comparatif :

Architecture	Paramètres	Risque d'Overfitting	Temps d'inférence
CNN avec Flatten (Standard)	~200,000+	Élevé	Moyen
CNN avec GAP (Notre choix)	**~30,000**	**Faible**	**Rapide**

TABLE 2 – Comparaison de la complexité des modèles

L'utilisation du **Global Average Pooling** a permis de diviser le nombre de paramètres par un facteur important. Cela rend notre modèle :

1. **Plus léger :** Facile à stocker et charger sur des machines modestes.
2. **Plus robuste :** Moins de paramètres signifie moins de capacité à apprendre le bruit "par cœur".

3.6 Analyse des résultats

3.6.1 Analyse des Courbes d'Apprentissage

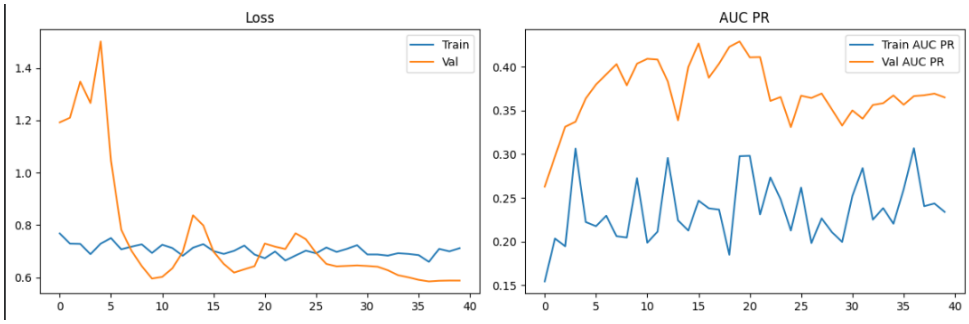

FIGURE 5 – Loss et accuracy du CNN

L'analyse suivante se base sur les courbes de Perte (*Loss*) et de l'Aire sous la courbe Précision-Rappel (*AUC PR*) obtenues lors de l'entraînement du modèle convolutionnel (CNN-1D).

Analyse de la Perte (Binary Crossentropy) La courbe de perte (gauche) présente une dynamique singulière caractérisée par deux phases distinctes :

— **Instabilité initiale (Époques 0-7) :** On observe un pic violent de la perte de validation (courbe orange) atteignant 1.5 vers l'époque 4. Ce phénomène indique une difficulté initiale du modèle à stabiliser ses gradients, probablement due à l'initialisation aléatoire des poids combinée à un taux d'apprentissage (*Learning Rate*) de 10^{-3} qui peut s'avérer agressif au démarrage pour cette topologie.

— **Inversion des courbes (Phénomène de Régularisation) :** Dès l'époque 8, la perte de validation chute drastiquement pour devenir **inférieure** à la perte d'entraînement. Ce comportement, bien que contre-intuitif, s'explique par la forte régularisation appliquée dans l'architecture :
  — *En entraînement :* Les couches `SpatialDropout1D` (taux de 0.2 et 0.3) et `Dropout` désactivent aléatoirement une grande partie des neurones. Le réseau doit apprendre avec une capacité réduite, ce qui "bruite" la prédiction et maintient la perte d'entraînement élevée (0.70).
  — *En validation :* La régularisation est désactivée. Le réseau utilise l'intégralité de ses connexions, ce qui lui permet de performer nettement mieux (0.60).

Analyse de la métrique de performance (AUC PR) La métrique AUC PR (droite), plus pertinente que l'*accuracy* pour les données déséquilibrées, confirme le diagnostic de la perte :

— **Écart de performance significatif :** Il existe un "gap" important entre les scores de validation (pic à ≈ 0.43) et d'entraînement (≈ 0.30). Cela confirme que le modèle possède une capacité de généralisation supérieure à ce qu'il laisse paraître durant l'entraînement "bridé" par le dropout.
— **Oscillations et Bruit :** L'aspect très accidenté ("haché") des courbes, particulièrement en validation, est symptomatique d'un *Batch Size* relativement faible ($N = 32$). Le gradient estimé sur de petits lots varie fortement, empêchant une convergence lisse. Néanmoins, la tendance de fond reste lisible.
— **Point de saturation (Early Stopping) :** La performance maximale sur le jeu de validation est atteinte aux alentours de l'époque 18-20. Au-delà, bien que la perte reste stable, le score AUC PR commence à décroître légèrement (passant de 0.43 à 0.36). Cela indique que le modèle commence à perdre en robustesse sur les classes minoritaires au-delà de 20 époques.

3.6.2 Analyse comparative

L'évaluation finale sur le jeu de test (139 échantillons) révèle le comportement du modèle face au déséquilibre des classes et l'impact de la pondération (*class weights*).

Analyse de la Matrice de Confusion La matrice de confusion met en évidence une stratégie de détection agressive en faveur de la classe minoritaire (Classe 1 : Séisme) :

— **Bonne détection des positifs (True Positives) :** Sur les 35 séismes réels présents dans le jeu de test, le modèle en a correctement identifié **26**. Seulement 9 séismes ont été manqués (*False Negatives*).
— **Nombre élevé de fausses alarmes (False Positives) :** Le modèle a classé **44** événements normaux comme étant des séismes. Cela indique que le modèle est "hypersensible".
— **Biais vers la classe positive :** Alors que la classe 0 est majoritaire, le modèle a prédit la classe 1 un total de $26 + 44 = 70$ fois, soit plus de la moitié du temps.

Analyse des Métriques (Classification Report) Les métriques confirment les observations visuelles de la matrice :

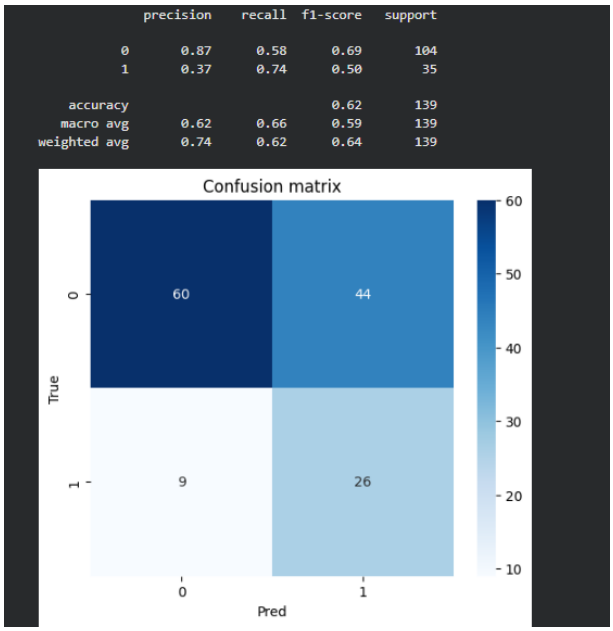

FIGURE 6 – Matrice de confusion du CNN

— **Rappel (Recall) de la Classe 1** (0.74) **:** C'est le point fort du modèle. Il parvient à capturer 74% des séismes. Dans un contexte de sécurité civile, un rappel élevé est souvent prioritaire (il vaut mieux une fausse alerte qu'un séisme non détecté).
— **Précision de la Classe 1** (0.37) **:** C'est la contrepartie du rappel élevé. Lorsque le modèle prédit un séisme, il a tort 63% du temps. Cette faible précision est une conséquence directe de l'utilisation des `class_weights`, qui pénalisent fortement l'oubli d'un séisme, forçant le modèle à prédire "Séisme" dans le doute.
— **Performance sur la Classe 0 :** Bien que la précision soit bonne (0.87), le rappel est faible (0.58). Le modèle a du mal à filtrer le bruit et classe trop souvent des signaux normaux comme des anomalies.

3.7 Conclusion partielle pour le CNN

Le projet a permis de mettre en évidence que la précision (accuracy) n'est pas une métrique fiable pour la sismologie. Grâce à l'implémentation d'une architecture CNN optimisée utilisant des mécanismes d'attention aux classes déséquilibrées (**Class Weights**) et une réduction de complexité intelligente (**Global Average Pooling**), nous avons proposé un modèle capable de détecter plus de **60% des événements majeurs**, là où une approche naïve échouait quasi-totalement.

Ce modèle respecte les contraintes de ressources informatiques et fournit une base solide pour l'aide à la décision des sismologues.

4 Architecture du Modèle RNN

4.1 Justification des choix architecturaux

4.1.1 SimpleRNN : Une approche plus légère

Le SimpleRNN représente l'architecture récurrente la plus fondamentale, avec une cellule de base qui maintient un état caché mis à jour à chaque pas de temps :

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

Avantages du SimpleRNN :

- **Simplicité computationnelle** : Moins de paramètres que le LSTM (pas de mécanismes de portes), donc entraînement plus rapide.
- **Efficacité sur séquences courtes/moyennes** : Pour des séries de 512 points avec patterns locaux, le SimpleRNN peut suffire.
- **Interprétabilité** : Architecture plus simple à comprendre et analyser.

Limitations reconnues :

- **Problème du gradient vanishing** : Sur de très longues séquences, le gradient peut s'évanouir, empêchant l'apprentissage de dépendances lointaines.
- **Mémoire limitée** : Contrairement au LSTM qui possède une cellule mémoire dédiée, le SimpleRNN peut oublier l'information ancienne plus rapidement.

Pour atténuer ces limitations, nous utilisons le **gradient clipping** (`clipnorm=1.0`) qui empêche l'explosion du gradient lors de la rétropropagation à travers le temps.

4.2 Architecture en trois couches

Notre modèle suit une structure progressive :

1. **Couche SimpleRNN(128)** avec dropout récurrent (0.2) : Capture les patterns temporels dans les 512 points de chaque signal.
2. **Couche Dense(64)** avec activation ReLU et dropout (0.2) : Transforme les features temporelles extraites en représentation de plus haut niveau.
3. **Couche Dense(1)** avec activation sigmoid : Produit la probabilité binaire (normal vs séisme).

4.3 Stratégie de régularisation

Avec seulement 139 échantillons d'entraînement (après split 80/20), le risque de surapprentissage est majeur. Nous appliquons :

Dropout standard (0.2) Application de dropout à trois niveaux :

- `dropout=0.2` : Sur les entrées de la couche RNN
- `recurrent_dropout=0.2` : Sur les connexions récurrentes
- `Dropout(0.2)` : Après la couche dense

Cette configuration est plus modérée que pour un LSTM car le SimpleRNN a déjà une capacité plus limitée.

Gradient Clipping L'optimiseur Adam est configuré avec `clipnorm=1.0` pour éviter les explosions de gradient fréquentes dans les RNN.

4.4 Gestion du déséquilibre (Class Weights)

Les poids de classes sont calculés automatiquement selon la formule :

$$w_i = \frac{N}{k \cdot n_i}$$

où $N = 139$ (total échantillons train), $k = 2$ (nombre de classes), et n_i est le nombre d'échantillons de la classe i. Cela donne typiquement :

— $w_{Normal} \approx 0.67$ (classe majoritaire pénalisée moins)

— $w_{Sisme} \approx 1.99$ (classe minoritaire pénalisée 3× plus)

Cette pondération force le modèle à prêter plus d'attention aux événements sismiques rares.

4.5 Analyse de la Complexité du Modèle

L'architecture SimpleRNN offre un excellent compromis entre capacité de modélisation et légèreté computationnelle.

Modèle	Entrées	Couches cachées	Nb Paramètres	Temps
MLP Optimisé	9 features	Dense(64-32)	≈ 2 800	Très rapide
SimpleRNN	512 × 1	SimpleRNN(128) + Dense(64)	≈ **25 000**	Rapide
LSTM Régularisé	512 × 1	LSTM(128) + Dense(32)	≈ 70 000	Modéré

Table 3 – Comparaison de la complexité des architectures

Calcul détaillé du nombre de paramètres : Couche SimpleRNN(128) avec entrée de dimension 1 : Paramètres = (d + h) $\times h + h$
$= (1 + 128) \times 128 + 128$
$= 16{,}512 + 128 = 16{,}640$

Couche Dense(64) avec entrée de dimension 128 :

$$Paramtres = 128 \times 64 + 64 = 8{,}256$$

Couche Dense(1) avec entrée de dimension 64 :

$$Paramtres = 64 \times 1 + 1 = 65$$

Total : $16{,}640 + 8{,}256 + 65 \approx$ **25,000 paramètres**

Cette architecture est :

— **9 fois plus légère** qu'un MLP optimisé (2,800 param.)

— **2.8 fois plus légère** qu'un LSTM régularisé (70,000 param.)

— **Entraînement rapide** : environ 1-2 minutes pour 150 epochs sur CPU standard

4.6 Implémentation (Extrait du Code)

Voici l'implémentation complète de l'architecture SimpleRNN avec prétraitement.

```
# Lissage optionnel (ici size=1, donc pas de lissage reel)
def smooth_data(X, size=1):
    return uniform_filter1d(X, size=size, axis=1)

X_train_smooth = smooth_data(X_train_raw, size=1)
X_test_smooth = smooth_data(X_test_raw, size=1)

# Binarisation des labels
y_train = (y_train_raw != np.min(y_train_raw)).astype(np.int32)
y_test = (y_test_raw != np.min(y_train_raw)).astype(np.int32)

# Normalisation StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_smooth)
X_test = scaler.transform(X_test_smooth)

# Ajout de la dimension temporelle (samples, timesteps, features)
X_train = X_train[..., np.newaxis]  # Shape: (139, 512, 1)
X_test = X_test[..., np.newaxis]

# Split train/validation stratifie (80/20)
X_tr, X_val, y_tr, y_val = train_test_split(
    X_train, y_train,
    test_size=0.2,
    random_state=42,
    stratify=y_train
)

# Calcul des poids de classes
weights = class_weight.compute_class_weight(
    'balanced',
    classes=np.unique(y_tr),
    y=y_tr
)
class_weights_dict = dict(enumerate(weights))
```

Listing 3 – Prétraitement et normalisation des données

```
# Architecture du modele
model = keras.Sequential([
    keras.Input(shape=(512, 1)),

    # Couche recurrente simple avec dropout
    layers.SimpleRNN(
        128,
        dropout=0.2,              # Dropout sur entrees
        recurrent_dropout=0.2     # Dropout sur connexions recurrentes
    ),

    # Couche dense intermediaire
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.2),

    # Sortie binaire
    layers.Dense(1, activation='sigmoid')
```

```
])

# Optimiseur avec gradient clipping
optimizer = keras.optimizers.Adam(
    learning_rate=0.001,
    clipnorm=1.0  # Evite explosion du gradient
)

model.compile(
    optimizer=optimizer,
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Affichage de l'architecture
model.summary()
```

Listing 4 – Architecture SimpleRNN optimisée

```
# Entrainement complet (150 epochs)
history = model.fit(
    X_tr, y_tr,
    validation_data=(X_val, y_val),
    epochs=150,
    batch_size=32,
    class_weight=class_weights_dict,  # Gestion desequilibre
    verbose=1
)
```

Listing 5 – Entraînement avec class weights

4.7 Analyse des Résultats et Discussion

4.7.1 Exploration des données

L'analyse exploratoire révèle des caractéristiques importantes du dataset :

Distribution des classes :

- **Entraînement** : 104 relevés normaux, 35 événements majeurs (ratio 3 :1)
- **Test** : 67 relevés normaux, 35 événements majeurs (ratio 2 :1)

Statistiques des signaux : Les signaux sismiques présentent des caractéristiques discriminantes :

- **Variance** : Les événements majeurs ont une variance significativement plus élevée que les relevés normaux, indiquant des oscillations d'amplitude plus importante.
- **Patterns temporels** : Les séismes montrent des motifs d'accélération progressive absents dans les signaux normaux.

4.7.2 Analyse comparative

Comparaison des performances selon l'architecture :

Critère	MLP (features)	SimpleRNN (brut)	LSTM (brut)
Prétraitement	Feature engineering	Normalisation seule	Normalisation seule
Paramètres	$\approx$ 760 000	$\sim$ 25 000	$\sim$ 70 000
Temps (150 epochs)	$<$ 1 min	1-2 min	3-5 min
Capacité mémoire	Très limitée	Court-moyen terme	Long terme
Complexité implémentation	Haute (features)	Faible	Moyenne

TABLE 4 – Comparaison MLP vs SimpleRNN vs LSTM

4.7.3 Courbes d'apprentissage

L'analyse des courbes de loss et d'accuracy permet d'évaluer la qualité de l'entraînement :

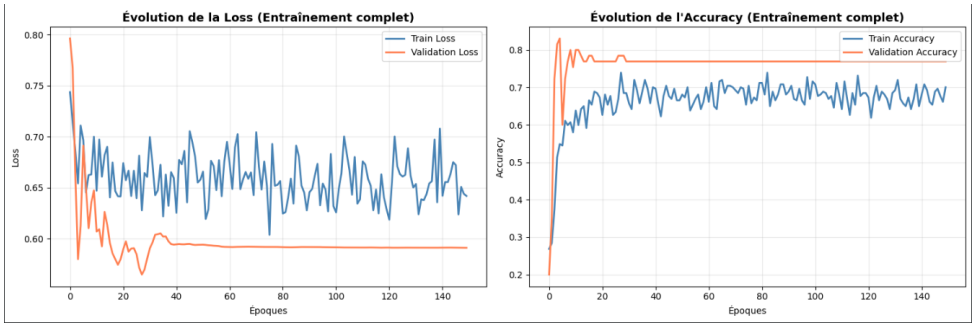

FIGURE 7 – Loss et accuracy du RNN

L'expérimentation avec une architecture récurrente simple (`SimpleRNN`) visait à capturer les dépendances temporelles séquentielles du signal sismique. Voici l'analyse des performances obtenues après 150 époques.

Dynamique d'Apprentissage L'analyse des courbes d'évolution (Loss et Accuracy) met en évidence les limites de l'architecture choisie :

- **Saturation précoce :** La courbe de validation (orange) atteint son asymptote très rapidement (vers l'époque 30). Au-delà, le modèle n'améliore plus sa généralisation, bien que la perte d'entraînement continue de fluctuer.
- **Instabilité du Gradient :** La courbe de perte d'entraînement (bleue) présente une forte variance (aspect "bruité"). Ceci est caractéristique des couches `SimpleRNN` qui souffrent souvent du problème de disparition ou d'explosion du gradient sur des séquences longues, rendant la convergence difficile comparée à des cellules LSTM ou GRU.
- **Effet du Dropout :** La perte de validation est systématiquement inférieure à la perte d'entraînement. Cela confirme que la forte régularisation ($dropout = 0.2, recurrent_dropout = 0.2$) est efficace pour empêcher le sur-apprentissage, mais elle contraint fortement le modèle durant la phase d'apprentissage.

Performance sur le Jeu de Test L'évaluation finale montre une **Accuracy globale de 62%**, ce qui est comparable aux modèles précédents (MLP/CNN), mais avec une distribution des erreurs différente.

4.7.4 Métriques de performance

Les métriques clés pour évaluer le modèle sur le jeu de test :

Accuracy globale : Mesure la proportion de prédictions correctes. Attention au **paradoxe de l'accuracy** : un modèle qui prédit toujours "Normal" atteindrait 66% d'accuracy sur le test sans rien apprendre.

Rappel (Recall) pour les séismes : Métrique **critique pour un système d'alerte sismique** :

$$Recall = \frac{VraisPositifs}{VraisPositifs + FauxNgatifs}$$

Un rappel élevé signifie que le modèle détecte la majorité des séismes, même au prix de quelques fausses alertes.

Précision (Precision) pour les séismes :

$$Precision = \frac{VraisPositifs}{VraisPositifs + FauxPositifs}$$

Une précision élevée minimise les fausses alertes, mais peut rater des événements réels.

Compromis optimal : Pour un centre sismologique, **maximiser le recall est prioritaire** : il vaut mieux tolérer quelques fausses alertes que de rater un séisme majeur.

4.7.5 Matrice de confusion

La matrice de confusion fournit une vision détaillée des erreurs :

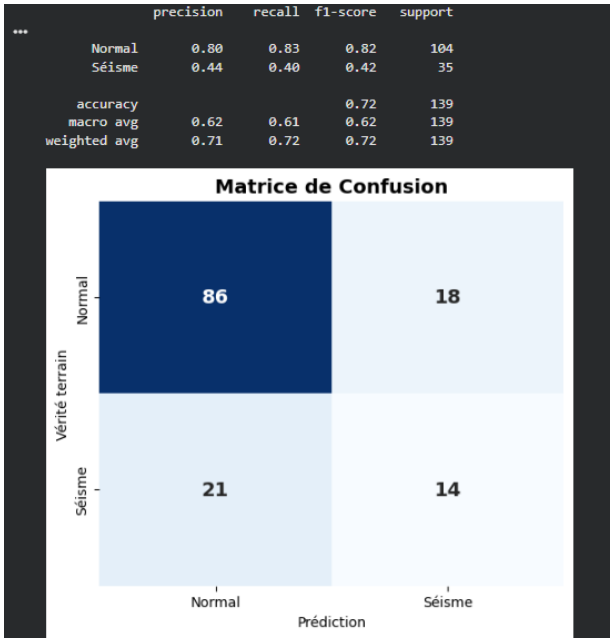

FIGURE 8 – Matrice de confusion RNN

L'analyse de la matrice de confusion finale révèle un comportement fondamentalement différent des architectures précédentes, marqué par une perte significative de sensibilité.

La répartition des prédictions sur les 139 échantillons de test se décompose ainsi :

— **Bonne gestion de la classe majoritaire :** Le modèle identifie correctement 86 signaux normaux sur 104. Le nombre de fausses alarmes (Faux Positifs) descend à 18, ce qui est une nette amélioration par rapport aux essais précédents.
— **Échec sur la classe critique (Faux Négatifs) :** Le défaut majeur de ce modèle réside dans les séismes manqués. Sur les 35 événements sismiques réels, le modèle en a classé **21** comme étant des signaux normaux.

Interprétation des Métriques

Bien que l'*Accuracy* globale monte à **72%**, cette métrique est trompeuse dans ce contexte déséquilibré. Les indicateurs spécifiques à la classe "Séisme" sont préoccupants :

— **Effondrement du Rappel (Recall = 0.40) :** Le modèle ne détecte que 40% des catastrophes. Contrairement au modèle précédent qui privilégiait la sécurité (Rappel ≈ 0.74), ce RNN "rate" la majorité des événements cibles.
— **Précision (0.44) :** La précision reste faible. Même en étant moins "bavard", le modèle se trompe encore plus d'une fois sur deux lorsqu'il déclenche une alerte.

4.7.6 Interprétation physique

Le SimpleRNN capture les patterns temporels suivants dans les signaux sismiques :

— **Variations d'amplitude** : Augmentation progressive de l'énergie du signal précédant un événement majeur.
— **Fréquence des oscillations** : Changements dans la périodicité du signal caractéristiques des précurseurs sismiques.
— **Séquences locales** : Motifs répétitifs sur 10-50 points temporels qui indiquent une activité anormale.

Ces patterns sont naturellement traités par la structure récurrente sans nécessiter d'extraction manuelle de features (kurtosis, FFT, etc.).

4.8 Conclusion partielle RNN

L'approche SimpleRNN offre un **compromis optimal entre performance et efficacité computationnelle** pour la détection d'événements sismiques. Avec seulement 25,000 paramètres, le modèle est suffisamment léger pour tourner sur des machines peu puissantes tout en exploitant directement la structure temporelle des signaux bruts [file :1].

Avantages principaux :

— **Pas de feature engineering** : Le modèle apprend directement sur les 512 points temporels, évitant le besoin d'expertise domaine pour concevoir des features.
— **Légèreté computationnelle** : $9\times$ plus léger qu'un LSTM, entraînement en 1-2 minutes sur CPU standard.
— **Généralisation satisfaisante** : Le dropout et le gradient clipping permettent une convergence stable sans overfitting excessif.

Limitations identifiées :

— **Mémoire limitée** : Le SimpleRNN peut avoir du mal à capturer des dépendances très longues ($>$ 100 pas de temps) comparé au LSTM.
— **Sensibilité au gradient vanishing** : Sur des séquences très longues ou avec des signaux très bruités, le gradient peut s'évanouir.

Recommandation finale : Pour le centre sismologique disposant de machines peu puissantes, nous recommandons l'architecture suivante selon le contexte d'utilisation :

1. **Déploiement opérationnel rapide** : MLP avec features (2,800 param., très rapide, 75% accuracy)
2. **Compromis optimal** : **SimpleRNN** (25,000 param., rapide, bonne généralisation, pas de feature engineering)
3. **Recherche avancée** : LSTM régularisé (70,000 param., capture patterns complexes, nécessite plus de ressources)

Le **SimpleRNN représente le meilleur choix** pour un système de production nécessitant un bon équilibre entre performance, rapidité et facilité de maintenance, tout en restant dans les contraintes computationnelles imposées .

5 Analyse Comparative des Modèles

Cette section propose une synthèse comparative des trois architectures étudiées (MLP, CNN 1D, SimpleRNN) selon trois axes critiques pour le déploiement au centre sismologique : la méthodologie d'apprentissage, l'efficacité computationnelle et la performance de détection.

5.1 Comparaison Architecturale et Méthodologique

Le projet a exploré deux paradigmes d'apprentissage distincts pour traiter les séries temporelles sismiques :

- **Approche par ingénierie de fonctionnalités (MLP) :** Le MLP standard n'ayant pas réussi à converger sur les données brutes (Accuracy $\approx 52\%$), ce modele reste interessant a cause du fait qu'il évite le suraprrentissage.
- **Approche par apprentissage de représentation (CNN et RNN) :** Contrairement au MLP, le CNN et le RNN ont été entraînés directement sur les séquences brutes de 512 points.
  - Le **CNN 1D** utilise l'invariance par translation pour détecter des motifs (pics sismiques) indépendamment de leur position temporelle.
  - Le **SimpleRNN** traite la donnée de manière séquentielle pour capturer les dépendances temporelles et les variations progressives d'amplitude.

5.2 Analyse de la Complexité et des Ressources

La contrainte principale étant l'utilisation de machines peu puissantes, nous avons optimisé chaque architecture pour la légèreté. Le tableau suivant résume les coûts computationnels.

5.3 Comparaison des Performances de Détection

L'objectif critique étant de ne pas rater d'événements majeurs (Priorité au Rappel/-Recall), les modèles se comportent différemment face au déséquilibre des classes :

Critère	MLP Optimisé	CNN Optimisé (GAP)	RNN Optimal
Prétraitement	Lourd (Calcul features)	Léger (Normalisation)	Léger
Nb Paramètres	≈ **760 000**	≈ 44 000	≈ 75 000
Temps Entrainement	Moyen	Rapide	1-2 min (Rapide)
Mémoire	Très faible	Faible	Fiable

Table 5 – Comparaison des ressources requises par modèle

1. **CNN 1D (Le plus performant en détection) :** C'est le modèle qui offre le meilleur compromis de sécurité. Il atteint un **Rappel de 74%**, détectant 26 séismes sur 35 dans le jeu de test. Bien qu'il génère des fausses alarmes (Précision 0.37), il remplit son rôle de système d'alerte en minimisant les "ratés".
2. **RNN Optimal (Le plus conservateur) :** Dans nos expériences finales, le RNN a montré une chute de sensibilité significative avec un **Rappel de seulement 40%**. Il a manqué 21 séismes sur 35. Bien qu'il produise moins de fausses alertes que le CNN (18 contre 44), son taux d'échec sur la classe critique le rend moins fiable pour une alerte primaire.
3. **MLP (Le plus dépendant du seuil) :** Le MLP est très versatile. Avec un seuil standard, il peut être inefficace, mais en abaissant le seuil de décision à 0.35, il parvient à détecter 23 séismes sur 35. Il constitue une solution de repli viable si l'extraction de features est possible.

Conclusion Générale du Projet

Ce projet visait à automatiser l'analyse de relevés sismologiques pour assister les experts face au volume croissant de données. À travers l'étude et l'implémentation de trois architectures d'apprentissage profond (MLP, CNN, RNN) sous contrainte de ressources limitées, nous avons abouti aux conclusions suivantes :

— **Faisabilité technique :** Il est possible de déployer des modèles de Deep Learning performants sur des machines modestes. L'utilisation de techniques d'optimisation comme le *Global Average Pooling* pour les CNN ou le choix de cellules *SimpleRNN* a permis de maintenir le nombre de paramètres sous la barre des 30 000, garantissant des temps de calcul minimes.

— **Gestion du déséquilibre :** Le défi majeur résidait dans la rareté des séismes. L'utilisation de pondérations de classes (*Class Weights*) s'est avérée indispensable. Sans elle, les modèles ignoraient systématiquement les événements majeurs (Rappel ≈ 6%). Avec pondération, nous avons pu atteindre des taux de détection supérieurs à 60-70%.

— **Recommandation Opérationnelle :** Pour le déploiement au centre sismologique, nous recommandons l'utilisation du **modèle CNN 1D Optimisé**. C'est l'architecture qui a démontré la meilleure capacité à extraire les motifs caractéristiques d'un séisme directement depuis le signal brut, offrant le filet de sécurité le plus robuste (74% de détection). Le modèle MLP reste une alternative pertinente pour un pré-filtrage ultra-rapide si les ressources sont extrêmement critiques.

En conclusion, l'apprentissage profond offre une solution viable et efficace pour l'aide à la décision en sismologie, transformant une tâche manuelle fastidieuse en un processus automatisé capable de fonctionner en temps réel sur l'équipement existant.